

LOADING UI(loading.tsx)

ở NextJS có một điểm rất khác so với React thông thường là tất cả component đầu là server component theo mặc định, nghĩa là server component này được render trực tiếp trên server

nếu là dự án React thuần thì trang HTML sau khi chạy hoàn toàn trống trơn, bởi đối với dự án React thuần mọi thứ được render từ phía trình duyệt, cho nên từ trình duyệt phải chạy JavaScript rồi mới có nội dung trên trang HTML

còn đối với NextJS thì trang HTML được server tạo sẵn rồi mới gửi xuống trình duyệt

mục tiêu bài học

hiểu được là component được chạy trên server cho nên developer có thể làm được những việc mà một dự án React thuần không làm được như:

- fetch data
- gọi database
- xử lý logic backend trong component

cách fetch() data từ API

tạo một biến để lưu user fetch về từ API

fetch() data cần sử dụng await nên cần async

tên hàm

```
const getUsers = async () => {  
  await delay(3000);  
  return fetch("https://jsonplaceholder.typicode.com/users").then((res)  
    res.json(),  
  );  
};
```

sau khi gọi hàm fetch("https:.....") để gửi http request, sau đó kết quả trả về sẽ là một promise ta sử dụng ".then()" để chờ response từ server rồi sau đó chuyển "res" đó sang "res.json" để port dữ liệu json thành javascript object

do biến users được sử dụng để lưu giá trị từ API gửi về cho nên sử dụng users.map() để map qua từng user và return về một thẻ li bên trong là {user.name} và phải thêm key và thẻ li

```
return (  
  <ul>  
    {users.map((user: any) => (  
      <li key={user.id}>{user.name}</li>  
    ))}  
  </ul>  
);
```

key

name

kết quả trả về của hàm sẽ là một thẻ ul chứa các thẻ li(Danh sách user được gửi về từ API)

có nghĩa là khi sử dụng hàm này ta phải truyền vào một tham số đầu vào với kiểu dữ liệu là number

tên hàm là delay

đối với hàm này ta sử dụng biến có tên là duration với kiểu dữ liệu number

```
const delay = (duration: number) => {  
  return new Promise((resolve) => setTimeout(resolve, duration));  
};
```

new Promise()

hàm này tạo một Promise, Promise này sẽ đại diện cho một nhiệm vụ sẽ được hoàn thành trong tương lai

bên trong Promise ta được cung cấp một function là resolve() hàm này là hàm gọi lại, nó giống như một chiếc nút khi nào xong thì nó bấm nút resolve này

sau duration ms thì sẽ gọi hàm resolve

ta nhận thấy nextJS catching data quá nhanh kể cả khi reload lại trang cho nên để trải nghiệm người dùng tốt hơn trong nextJS là loading state nếu developer cung cấp cho 1 file "loading.tsx"

lúc này khi reload lại trang những data gọi từ API trong thời gian đợi sẽ được thay bằng (return loading...) trong file loading.tsx rồi sau khi load xong loading... đó sẽ biến mất và thay vào đó là data được trả về